skeo

Claude: the full curriculum

How AI and Claude actually work, how to work with Claude well, and how to ship real work with it.

8	8	67	1
weeks	modules	lessons	capstone

Week 1 Foundations: How AI and Claude Actually Work

Week 2 Working With Claude Well: Prompting & Workspace

Week 3 The Visual & Creative Layer

Week 4 Reaching Further: Data, Tools & Research

Week 5 Delegation, Automation & Building

Week 6 Beyond the Individual: Devices, Teams & Enterprise

Week 7 Real-World Use Case Library

Week 8 Capstone Menu

WEEK 1 · M01

Foundations: How AI and Claude Actually Work

1.0 Module Intro

- What this module covers: how Claude actually works underneath, where it goes wrong, and which surface to reach for
- Why it comes first: every later module assumes you can tell fluent output from accurate output
- How each week runs: watch the lessons, try every task in Claude yourself, then take the module quiz to unlock the next week

1.1 What Generative AI Actually Is

- Claude predicts the next likely piece of text based on patterns learned from huge amounts of data, not facts pulled from a database
- Fluency (sounding right) and accuracy (being right) are separate properties: one doesn't guarantee the other
- Every AI interaction is a mix of machine properties (what the model has/lacks) and human competencies (what you bring to steer it well); this course teaches both together

1.2 Tokens, Context Windows & Why Claude Forgets

- Text is broken into tokens (word-pieces) before Claude processes anything
- The context window is the max tokens Claude can hold at once, up to 1M tokens on flagship models today, smaller on lighter ones
- Once a conversation exceeds that limit, older content gets pushed out or summarised, which is why long chats feel like Claude "forgot" something
- A brand-new chat always starts blank unless you're inside a Project that carries context forward

> **ASSIGNMENT**
> Run a long conversation, then ask Claude to recall your very first message. See what happened.

1.3 Hallucinations: Why They Happen, How to Catch Them

- A hallucination is confident, fluent, but factually wrong output: fake citations, invented statistics, wrong specifics
- Happens most on niche topics, exact numbers, quotes, and anything past the knowledge cutoff
- Fix: use web search for time-sensitive or fact-specific questions instead of relying on memory
- Ask directly, "are you sure, or could this be a guess?" Claude will often flag its own uncertainty when asked

> **ASSIGNMENT**
> Deliberately trigger a hallucination (ask about something obscure). Then verify it with search.

1.4 Steerability & Property Collisions

- Steerability is why the exact same question gets different answers depending on how you frame it: instructions genuinely change behaviour, not just tone
- Limitations don't show up in isolation; sometimes a short context window and a hallucination-prone gap collide on the same task, making the failure harder to diagnose
- Once you can name which property is misbehaving, you can fix it directly instead of just re-asking the same way and hoping

> **ASSIGNMENT**
> Take one task where Claude's output disappointed you. Name which property (memory, knowledge, hallucination, steerability) was actually the cause.

1.5 Claude OS: Chat, Projects, Artifacts, Cowork, Code

- Chat: quick questions, writing, analysis, back-and-forth
- Projects: persistent workspace that remembers context, instructions and files across every conversation inside it
- Artifacts: standalone reusable output (documents, code, diagrams, small interactive apps) that lives outside the chat scroll
- Cowork: a working session where Claude operates directly on your real files and folders, not just in a chat window
- Claude Code: an agentic coding partner across terminal, IDE, desktop and web
- Simple rule: quick question → Chat. Recurring context → Project. Something to keep/edit → Artifact. Multi-step real task on real files → Cowork. Building software → Claude Code

> **ASSIGNMENT**
> Run one task in plain Chat, then again inside a fresh Project. Compare the two outputs.

1.6 The Claude Model Family: Choosing the Right One

- Opus: deepest reasoning, for the hardest coding, research and enterprise work
- Sonnet: the balanced default for most everyday work
- Haiku: fastest and lightest, best for high-volume or simple tasks
- Mythos tier (Fable 5 / Mythos 5): sits above Opus, reserved for the very highest capability needs
- Practical rule: default to Sonnet, escalate to Opus/Mythos for genuinely hard problems, drop to Haiku when speed or volume matters more than depth

> **ASSIGNMENT**
> Run one task on two different models. Score speed, depth and usefulness for that specific task.

1.7 Memory: What Claude Remembers About You

- Separate from Project context, Memory is Claude retaining preferences and facts across chats generally
- You can view, edit or clear what's remembered at any time
- More continuity means more to actively manage for privacy, so it's a trade-off, not a free upgrade

> **ASSIGNMENT**
> Check what Claude currently remembers about you. Edit or remove one thing.

1.8 Extended Thinking & Effort

- Claude can "think longer" before answering on genuinely hard problems, visible or adjustable depending on the surface
- Higher effort = deeper reasoning and a slower response; not every question needs it
- Matching effort to task difficulty is itself a Delegation skill, covered fully in Module 2

> **ASSIGNMENT**
> Run the same hard problem at two different effort/thinking levels. Compare the quality gap.

Module 1 Recap

- A run-back through the whole module before its quiz:
- 1.1 What Generative AI Actually Is
- 1.2 Tokens, Context Windows & Why Claude Forgets
- 1.3 Hallucinations: Why They Happen, How to Catch Them
- 1.4 Steerability & Property Collisions
- 1.5 Claude OS: Chat, Projects, Artifacts, Cowork, Code
- 1.6 The Claude Model Family: Choosing the Right One
- 1.7 Memory: What Claude Remembers About You
- 1.8 Extended Thinking & Effort

WEEK 2 · M02

Working With Claude Well: Prompting & Workspace

2.1 The 4D Framework

- Delegation: deciding what to hand to Claude and what to keep for yourself; not everything should be delegated
- Description: giving context, constraints, audience and desired outcome, not just a bare question
- Discernment: reading Claude's output critically before you trust or act on it
- Diligence: staying accountable for whatever you ship; Claude doesn't carry that responsibility, you do
- Description and Discernment form a loop (describe, evaluate, refine, repeat), not a one-shot request

> **ASSIGNMENT**
> Take 5 real prompts you've used. Rewrite each with genuine Description. Compare before/after.

2.2 Prompt Engineering: The Craft Underneath Description

- Vague prompts produce vague output: this is the garbage-in rule, and it's the single biggest lever you control
- Core levers to pull: context, length/format, examples, audience, and the specific result you want
- Role prompting ("act as a senior analyst") sharpens tone and framing but doesn't add knowledge Claude doesn't have
- Constraints are quality tools, not restrictions: word limits, format rules and tone directives all narrow the output toward what you actually need
- Chain prompting: break a complex task into sequential steps instead of one giant ask; each step's output becomes the next step's input
- Iterative refinement: treat Claude as a collaborator across multiple turns, not a vending machine you get one shot at
- For technical or highly structured asks, giving a clear example of the format you want beats describing the format in words
- What not to do: over-specifying trivial tasks (wastes your time), under-specifying important ones (wastes Claude's accuracy)

> **ASSIGNMENT**
> Take one prompt that gives a weak result. Rewrite it 3 progressively stronger times. Compare all three outputs.

2.3 Prompt Injection Awareness

- A prompt injection is an instruction hidden inside a document, webpage or file that tries to hijack what Claude does next
- The core distinction: instructions embedded in content you asked Claude to read are not the same as instructions from you directly
- Red flag pattern: unexpected commands appearing inside "data" Claude is processing, rather than in your own message

> **ASSIGNMENT**
> Read one real example of a prompt injection attempt online. Identify the exact red flag that gives it away.

2.4 Projects: Your Persistent Workspace

- A Project keeps context, uploaded files and custom instructions in place across every conversation inside it
- As a project's files approach the model's context limit, Claude automatically switches to retrieval mode, searching and pulling only relevant passages instead of hitting a hard wall
- Custom instructions define who Claude should be inside this Project: tone, assumptions, default output format
- Practical pattern: one Project per role or domain (Research, Client Work, Study) rather than one giant catch-all

> **ASSIGNMENT**
> Build a Project with real instructions (150+ words), one uploaded document, and 3 completed real tasks.

2.5 Artifacts: Turning Chat Into Real Deliverables

- Standalone output opens in its own panel: documents, code, HTML pages, SVG diagrams, Mermaid flowcharts, small interactive tools
- Editable in place: highlight a section, describe the change, Claude edits just that part
- Can hold a small amount of persistent data, enough for a simple tracker or checklist
- Habit to build: don't let good output die in the chat scroll; if you'll reuse or share it, make it an Artifact

> **ASSIGNMENT**
> Build one real deliverable as an Artifact through at least two rounds of iteration, not one shot.

2.6 Skills: Teaching Claude Your Way, Once

- A Skill is a reusable instruction set Claude applies automatically whenever a task matches its description
- Costs almost no context until triggered, so you can have many installed and only pay for the ones actually used
- Can carry reference material (a style guide, a checklist), accept inputs, and chain to other skills

> **ASSIGNMENT**
> Write a Skill for one output you produce often. Test it on a raw input with the Skill off, then on.

2.7 Plugins: Packaged Expertise

- A Plugin bundles Skills, Connectors and sometimes Subagents into one installable package
- How a team encodes a shared standard instead of everyone configuring their own version
- Works in both chat and Cowork; features needing an isolated session (hooks, subagents) only run in Cowork

> **ASSIGNMENT**
> Install one relevant Plugin. Run one real task through it and note what it set up automatically.

2.8 Troubleshooting

- A Skill that won't trigger is usually a vague description problem, not a broken tool
- If a Connector fails, check authorisation and permissions before assuming something's wrong with the connector itself
- Read Claude's own clarifying or error messages carefully before assuming a task failed outright

> **ASSIGNMENT**
> Deliberately weaken your Skill's description from 2.6. Diagnose exactly why it stopped firing.

2.9 Sharing, Exporting & Publishing Your Work

- Artifacts can be exported, shared via link, or published depending on the surface and your plan
- Know the difference between "keep it private", "share with my team" and "publish publicly" before you click anything
- Reviewing before publishing is Diligence in practice, the last human check before something leaves your hands

> **ASSIGNMENT**
> Take one Artifact from this course and go through the full export/share flow once, end to end.

Module 2 Recap

- A run-back through the whole module before its quiz:
- 2.1 The 4D Framework
- 2.2 Prompt Engineering: The Craft Underneath Description
- 2.3 Prompt Injection Awareness
- 2.4 Projects: Your Persistent Workspace
- 2.5 Artifacts: Turning Chat Into Real Deliverables
- 2.6 Skills: Teaching Claude Your Way, Once
- 2.7 Plugins: Packaged Expertise
- 2.8 Troubleshooting
- 2.9 Sharing, Exporting & Publishing Your Work

WEEK 3 · M03

The Visual & Creative Layer

3.0 Module Intro

- What this module covers: Claude reading images, drawing diagrams, charts and interactive Artifacts, and polishing visuals in Claude Design
- Why it matters: a picture or a working tool often explains more than a paragraph ever could
- What you will make: real visual output, from a clean note off a whiteboard photo to a small build you can share

3.1 Claude and Images: Reading, Not Just Writing

- Claude accepts image input: screenshots, photos, scanned documents, charts, whiteboards
- Use case: a photo of handwritten or whiteboard notes → a clean, structured document
- Use case: a screenshot of an error → Claude reads and explains it directly, no retyping needed

> **ASSIGNMENT**
> Feed Claude a photo of handwritten or whiteboard notes. Get a structured summary back.

3.2 Claude and Visual Output: Diagrams, Charts, Interactive Artifacts

- Claude renders SVG graphics, Mermaid diagrams and interactive HTML natively inside Artifacts
- A spreadsheet can become a live, clickable dashboard instead of a static chart
- Ask for a diagram of a concept instead of a paragraph explaining it, often clearer, always faster to scan
- Important distinction: Claude generates diagrams and interactive visuals natively, but does not natively generate photorealistic images, audio or video, which require a connected specialist tool

> **ASSIGNMENT**
> Take one concept you'd normally explain in text. Ask Claude to diagram it instead. Compare clarity.

3.3 Claude Design: Prototypes, Mockups & Polished Visuals

- A dedicated Claude app for turning ideas into slides, prototypes, landing pages and one-pagers
- Best fit: pitch decks, visual mockups and polished one-page documents that need real design sense, not just structure
- Pairs naturally with Artifacts: draft the structure in chat, polish the visual layer in Claude Design

> **ASSIGNMENT**
> Take one Artifact from Module 2 and rebuild its visual layer using Claude Design.

3.4 Working With Video & Audio Content

- Claude reasons over transcripts, meeting notes and captions you provide: it processes the text of spoken content, not the raw audio/video file as media
- Feed in a voice memo transcript, get a structured brief and a first-draft email out, in one pass
- For actual image, video or audio generation, Claude connects out to specialist tools rather than producing that media itself

> **ASSIGNMENT**
> Take a real or recorded transcript and turn it into a structured brief in a single prompt.

3.5 Bonus Round: Fun, Impressive Builds

- A quiz, trivia game or choose-your-own-adventure as an interactive Artifact, built in one prompt
- A simple browser game (tic-tac-toe, memory match) as a throwaway Artifact
- A working calculator, converter or planner tool generated from a plain description

> **ASSIGNMENT**
> Pick one bonus build and ship it in a single sitting, purely to see the range of what's possible.

Module 3 Recap

- A run-back through the whole module before its quiz:
- 3.1 Claude and Images: Reading, Not Just Writing
- 3.2 Claude and Visual Output: Diagrams, Charts, Interactive Artifacts
- 3.3 Claude Design: Prototypes, Mockups & Polished Visuals
- 3.4 Working With Video & Audio Content
- 3.5 Bonus Round: Fun, Impressive Builds

WEEK 4 · M04

Reaching Further: Data, Tools & Research

4.0 Module Intro

- What this module covers: connecting Claude to the tools you already use, cited research, Claude inside Chrome and Office, voice, other languages, and Cowork on real files
- Why it matters: the answer you need often lives outside the chat, in your files, your inbox and the web
- The habit to build: authorise each connection on purpose, and check the sources Claude brings back

4.1 Connectors & MCP

- Connectors let Claude read from and sometimes act on tools you already use: Drive, Gmail, Slack, DocuSign and more
- MCP (Model Context Protocol) is the open standard many connectors are built on, which is what lets the ecosystem grow over time
- You explicitly authorise each connection; nothing connects silently or by default

> **ASSIGNMENT**
> Connect one real tool. Run a task only that connection could answer, no manual copy-paste.

4.2 Research: Trustworthy, Cited Deep Dives

- Runs a multi-step, multi-source investigation and returns a cited answer you can trace back to its sources
- The right tool when a plain chat answer would need independent verification anyway
- Discernment still applies: read the actual citations, not just the summary paragraph

> **ASSIGNMENT**
> Run a real research question. Build a 300-word cited brief with 3 concrete recommendations.

4.3 Claude Everywhere: Chrome, Excel, Word, PowerPoint, Outlook

- Claude in Chrome: a browsing agent that can navigate pages and act on your behalf
- Claude for Excel / Word / PowerPoint / Outlook: works directly inside the app you already use
- Removes the copy-paste tax between thinking with Claude and delivering the actual work

> **ASSIGNMENT**
> Complete one real task inside one of these apps, start to finish, without leaving it.

4.4 Voice Mode & Dictation

- Talking to Claude instead of typing, useful for brainstorming or hands-busy moments
- Dictation often beats typing when you're thinking out loud and don't want to interrupt the flow to type
- Precise technical asks still benefit from typed detail, since voice isn't always the faster path

> **ASSIGNMENT**
> Brainstorm one idea entirely by voice. Compare the result to a typed version of the same idea.

4.5 Multilingual Use

- Claude works across many languages, not only English
- Translation quality varies by language pair and context, so treat high-stakes translations with the same Discernment as any other fact
- Useful for drafting directly in a second language or checking tone and register before sending something out

> **ASSIGNMENT**
> Draft one message in a second language, then ask Claude to check tone and accuracy.

4.6 Claude Cowork: Working Sessions on Real Files

- A working session, not a conversation: Claude plans a multi-step task and executes it directly on your files
- Runs on the same agentic architecture as Claude Code, in an interface built for non-developers
- Can coordinate sub-agents on parallel pieces of one task, and use Chrome/Office extensions as tools mid-task
- Safety is built in: Cowork pauses on risky or irreversible actions, or asks first, rather than guessing

> **ASSIGNMENT**
> Hand Cowork one full multi-step task. Steer it at least once mid-task, then document the result.

Module 4 Recap

- A run-back through the whole module before its quiz:
- 4.1 Connectors & MCP
- 4.2 Research: Trustworthy, Cited Deep Dives
- 4.3 Claude Everywhere: Chrome, Excel, Word, PowerPoint, Outlook
- 4.4 Voice Mode & Dictation
- 4.5 Multilingual Use
- 4.6 Claude Cowork: Working Sessions on Real Files

WEEK 5 · M05

Delegation, Automation & Building

5.0 Module Intro

- What this module covers: subagents, scheduled and recurring tasks, building with Claude Code, and using all of it responsibly
- Why it matters: this is the step from asking Claude for answers to handing it whole pieces of work
- The habit to build: define exactly what should come back, and review everything before you ship it

5.1 Subagents: Delegating Without Losing Context

- An isolated Claude instance that works on one piece of a task independently, with its own context window
- Reports back only a summary, keeping your main conversation clean and focused
- Best for noisy, exploratory or parallelisable work where you only care about the outcome, not the process

> **ASSIGNMENT**
> Split one long task into a main task plus one delegated sub-task. Define exactly what should come back.

5.2 Scheduled & Recurring Tasks

- Turn a one-off Cowork task into a recurring one: a daily brief, a weekly review
- The prompt must be self-contained, since there's no back-and-forth mid-run to clarify anything
- Rule of thumb: exhaust native Claude automation before reaching for external tools like Zapier or n8n

> **ASSIGNMENT**
> Build a recurring brief. Run it for 3 cycles, refining the prompt after the first.

5.3 Claude Code: Building Real Things, No Deep Coding Required

- The loop: explore what exists → plan the change → let Claude write the code → review and commit
- You don't need to know how to code: describe the outcome clearly, review what comes back, ask for specific fixes
- Good fit: small internal tools, trackers, calculators, dashboards, landing pages
- Not a good fit without expert review: production-scale or security-sensitive systems

> **ASSIGNMENT**
> Build one small real tool. Iterate at least 3 times. Share it with someone outside this course.

5.4 Comparing Claude to Other AI Tools

- Not every task needs Claude: knowing when a specialist tool is the better fit is part of fluency, not a failure
- Claude's real strength is reasoning, writing, judgment and agentic multi-step work
- Other tools lead on pure image/video generation and narrow single-purpose apps

> **ASSIGNMENT**
> Name one task you currently force into Claude that a specialist tool would genuinely do better.

5.5 Responsible & Ethical Use

- Attribution and disclosure: being upfront about where AI assisted real work
- Academic integrity: using Claude to learn faster, not to bypass the learning itself
- Professional integrity: reviewing before you put your name on anything AI-assisted

> **ASSIGNMENT**
> Write your own one-line personal rule for when you'll disclose AI assistance, and when you won't need to.

5.6 Safety, Judgment & Career Positioning

- Diligence stays with you: review before sending, publishing, deleting or spending anything
- No safety layer, including Cowork's built-in checks, replaces your own judgment
- Proof of specific outcomes beats a list of tools used: "I built X, saving Y hours" over "I use AI"

> **ASSIGNMENT**
> Write 3–5 sentences describing how you actually use Claude, with one concrete outcome.

Module 5 Recap

- A run-back through the whole module before its quiz:
- 5.1 Subagents: Delegating Without Losing Context
- 5.2 Scheduled & Recurring Tasks
- 5.3 Claude Code: Building Real Things, No Deep Coding Required
- 5.4 Comparing Claude to Other AI Tools
- 5.5 Responsible & Ethical Use
- 5.6 Safety, Judgment & Career Positioning

WEEK 6 · M06

Beyond the Individual: Devices, Teams & Enterprise

6.0 Module Intro

- What this module covers: Claude on every device, API basics, team and enterprise features, data and privacy, role-based use, and choosing a plan
- Why it matters: the shift from your own use of Claude to how it fits a team or an organisation
- The habit to build: know what data goes where before you roll Claude out to anyone else

6.1 Claude on Every Device

- Claude mobile app (iOS/Android): Chat and Cowork on the go
- Claude Desktop: a unified app combining Chat, Cowork and Code
- Claude Tag: teammates tag @Claude into Slack to delegate work without switching apps

> **ASSIGNMENT**
> Complete one real task entirely from your phone.

6.2 Building on Claude: API Basics

- Covers system prompts and citation mechanics, and why exact wording in instructions matters at the API level

> **ASSIGNMENT**
> Optional: read one API quickstart. Note one thing that surprised you about how it works.

6.3 Team & Enterprise Layer

- Shared Projects, admin controls and plugin governance at the organisation level
- Group-based access means not everyone in an org sees the same plugin catalogue

> **ASSIGNMENT**
> If applicable, identify what your organisation’s admin controls or shares centrally.

6.4 Data, Privacy & Permissions

- What Claude can see, what it can’t, and how retention settings actually work
- Enterprise data controls differ meaningfully from personal account defaults

> **ASSIGNMENT**
> Review your own privacy and retention settings once, end to end.

6.5 Role-Based Claude

- Pre-built flavours for Engineering, Marketing, Sales, HR, Product and Financial Services
- A faster starting point than configuring a workspace completely from scratch

> **ASSIGNMENT**
> Try the role-based flavour closest to your work. Note what it pre-configured for you automatically.

6.6 Plans & Choosing the Right Tier

- Free, Pro, Max, Team and Enterprise plans differ in usage limits, model access and collaboration features
- Match the tier to actual usage patterns: heavy daily use and light occasional use have very different needs
- Revisit this choice periodically as your usage (and Claude's feature set) changes

> **ASSIGNMENT**
> Compare your current plan against your actual weekly usage. Note if it's under- or over-provisioned.

6.7 Staying AI-Literate

- Spotting AI-generated misinformation is now as practical a skill as producing content with AI
- Track new Claude releases without relearning everything from scratch each time: skim, don't re-study
- Media literacy has moved from an academic topic to a daily practical one

> **ASSIGNMENT**
> Find one piece of content online. Assess whether it's likely AI-generated, and explain why.

Module 6 Recap

- A run-back through the whole module before its quiz:
- 6.1 Claude on Every Device
- 6.2 Building on Claude: API Basics
- 6.3 Team & Enterprise Layer
- 6.4 Data, Privacy & Permissions
- 6.5 Role-Based Claude
- 6.6 Plans & Choosing the Right Tier
- 6.7 Staying AI-Literate

WEEK 7 · M07

Real-World Use Case Library

7.1 Personal Productivity

- Daily/weekly planning brief
- Inbox triage
- Meeting prep and follow-up
- Decision-making support
- Faster learning

7.2 Work: Knowledge & Office Tasks

- Report and memo drafting
- Decks from an outline
- Spreadsheet cleanup
- Research briefs
- File organisation

7.3 Students & Academics

- Study guides
- Practice questions
- Literature review scaffolding
- Multi-angle explanations

7.4 Builders & Creators

- Prototypes via Claude Code
- Content repurposing
- Dataset-to-insight projects

7.5 Small Business, Freelance & Ops

- Proposals and contracts
- Customer emails in your voice
- Expense cleanup
- One Project per business function

7.6 Automation & Systems Thinking

- The 3-task method
- Chained tool workflows
- Knowing when to leave Claude for external automation

7.7 Career & Portfolio

- Visible portfolio of real outputs
- Interview-ready workflow explanations
- Outcome-based positioning

7.8 Claude as Thinking Partner

- Debate mode
- Red-team your own plan
- Rubber-duck a stuck problem
- Perspective-shifting
- Pressure-test writing

Module 7 Recap

- A run-back through the whole module before its quiz:
- 7.1 Personal Productivity
- 7.2 Work: Knowledge & Office Tasks
- 7.3 Students & Academics
- 7.4 Builders & Creators
- 7.5 Small Business, Freelance & Ops
- 7.6 Automation & Systems Thinking
- 7.7 Career & Portfolio
- 7.8 Claude as Thinking Partner

WEEK 8 · M08

Capstone Menu

8.1 Personal AI Operating System

- A Project + Skill + Connector completing 3 real weekly tasks

8.2 Research Intelligence Brief

- A cited deep-dive on a real decision, using Research + Discernment

8.3 Automation Suite

- One Cowork task + one Skill + one recurring scheduled brief, running for real

8.4 Vibe-Built Tool

- A small working app or dashboard built with Claude Code, shared with one outside person

8.5 Visual Showcase

- A dashboard Artifact + a Claude Design piece + one "cool trick", combined into one demo

Module 8 Recap

- A run-back through the whole module before its quiz:
- 8.1 Personal AI Operating System
- 8.2 Research Intelligence Brief
- 8.3 Automation Suite
- 8.4 Vibe-Built Tool
- 8.5 Visual Showcase